🟦 1. TIME COMPLEXITY —

📌 What is Time Complexity?

Time complexity tells us **how much time an algorithm will take when the input becomes large.**

It does *not* measure actual seconds.
It measures **how the time grows** when input size grows.

👉 Think of it like this:

If you clean a room with **10 items**, it takes little time.
If you clean a room with **500 items**, it takes more time.

Same person → same speed → but time increases because **items increased**.

That's exactly what happens in coding too.

📌 Why do we need time complexity?

Because two programs can give the same output,
but one may run much faster than the other.

Example:
Program A checks every element one by one.
Program B jumps and checks smartly.

Both find the correct answer,
but **Program B is faster** — and we want to know *why* and *how much*.

Time complexity helps compare them.

📌 A simple coding example

Suppose you have an array of names.

To print all names, you do:

for each name → print

If names = 10 → 10 operations
If names = 1000 → 1000 operations

Operations grow **in proportion** with input.

This is called **O(n)** (read: "Order of n").

🟦 2. SPACE COMPLEXITY —

📌 What is Space Complexity?

Space complexity measures **how much extra memory** your program needs.

This includes:

- variables
- arrays
- temporary data
- recursion stack
- extra copies

👉 **Real-life example:**

Imagine you're making tea.

- If you make tea for 1 person → one cup
- For 10 people → more cups and a bigger pot

You need *extra space* depending on how many people (input size).

Similarly, your program may need:

- small memory for small input
- more memory for large input

Space complexity helps measure this memory usage.

📌 **Simple coding examples**

1️⃣ Using one variable → O(1) memory
2️⃣ Using an array of size n → O(n) memory
3️⃣ Using a 2D matrix of n × n → $O(n^2)$

🟩 3. Big O, Ω (Omega), Θ (Theta) —

These 3 are **mathematical notations** that describe algorithm performance.

To make it human:

✓ **Big O → Worst Case**

✓ **Omega (Ω) → Best Case**

✓ **Theta (Θ) → Average Case**

Let's explain all three with real life.

🟥 **Big O – Worst Case (The slowest it can get)**

Think: "How bad can things get?"

Real-Life Example:

You search for your missing key in the house.

Worst case?
👉 You search **every room**, **every drawer**, and **every pocket**.

You check *everything*.
This is **Big O** — the maximum effort.

Coding Example:

Searching an element in an unsorted array

Worst Case → element found at the last index
👉 **O(n)**

🟩 **Omega (Ω) – Best Case (Fastest it can be)**

Think: "Lucky case."

Real-Life Example:

You open the door, and the key is lying right there.
You didn't even search.

Best case = minimum effort.
That's **Ω**.

Coding Example:

Searching in an unsorted array

Best Case → element found at index 0
👉 **Ω(1)**

🟧 **Theta (Θ) – Average Case (Normal case)**

Think: "Usually what happens."

It’s neither extremely fast nor extremely slow.

Real-Life Example:

On average, you find your key after checking **2–3 rooms**, not instantly but not after checking the whole house either.

That’s Θ — typical behavior.

Coding Example:

Searching in random array
Average Case → element found in the middle
👉 **Θ(n/2)** but written as **Θ(n)**

Because we ignore small constants.

🟦 4. Best / Worst / Average Case

Let’s take the example of looking for your pen on a big study table.

✓ Best Case

You see it immediately → 1 step
→ **O(1)**

✓ Worst Case

It’s under some notebook at the corner
→ you check the whole table
→ **O(n)**

✓ Average Case

It may be somewhere in the middle
→ around half the table checked
→ **O(n)**

The point:
Best, worst, and average give a complete picture of algorithm behavior.

⭐ 5. Amortized Analysis —

This confuses many students, but it's actually simple.

Amortized analysis means:

👉 **Some operations are slow sometimes,**
but fast most of the time —
so average becomes fast.

📌 **Real-Life Example: Buying Bus Card Refill**

You have a travel card:

- Each day you tap → fast (1 sec)
- After 30 days → card expires → you recharge (slow)

Out of 30 days:

- 29 days = fast
- 1 day = slow

But overall, **average per day is fast**.

This is amortized behavior.

📌 **Coding Example: Dynamic Array / Vector / ArrayList**

When you insert elements:

- Most of the time → insert = O(1)
- Rarely, when full → array expands = O(n)

If you insert 1000 elements:

- 990 inserts are O(1)
- Only a few inserts are O(n)

So average = **O(1)**
This is called **Amortized O(1)**.

🟦 6. Different Types of Time Complexities (Detailed List)

Complexity	Meaning	Real-Life Example
O(1)	Constant time	Opening the first app on your phone
O(log n)	Divide & conquer	Finding a name in phone directory
O(n)	Linear time	Checking each item in a list
O(n log n)	Sorting algorithms	Sorting clothes by size
O(n^2)	Nested loops	Comparing each student with all others
O(2^n)	Exponential	Trying all combinations
O(n!)	Factorial	Arranging people in all possible orders

🟦 7. 10 Practice MCQs

1. Time complexity is used to measure:

A) Machine speed
B) Algorithm speed when input grows
C) Memory usage
D) Number of functions

Answer: B

---

2. Big O refers to:

A) Best case
B) Average case
C) Worst case
D) Space complexity

Answer: C

---

3. Ω notation tells:

A) Maximum time
B) Minimum time
C) Random time
D) Hardware usage

Answer: B

---

4. Θ gives:

A) Only worst-case time
B) Only best-case time
C) Tight bound (average)
D) Space complexity

Answer: C

---

5. Searching an element in the first position is:

A) O(1)
B) O(n)
C) O(n log n)
D) $O(n^2)$

Answer: A

---

6. Searching an element in the last position is:

A) Best case
B) Worst case
C) Constant time
D) None

Answer: B

---

7. Amortized analysis applies to:

A) Sorting arrays
B) Expanding dynamic arrays
C) Comparing strings
D) Loops

Answer: B

---

8. Which is fastest?

A) $O(n^2)$
B) O(n)

C) O(1)
D) O(log n)

Answer: C

---

9. Space complexity measures:

A) Extra memory used
B) Execution time
C) Output size
D) Code length

Answer: A

---

10. Which grows fastest?

A) O(1)
B) O(n)
C) O(log n)
D) $O(n^2)$

Answer: D